

Exploring the Integration of the Latest Version of vLLM with Ray 2.35.0: Gaps, Issues, and Challenges

2025-07-29 · marshall

最近大半年都在做AI Infra相关的学习，也算是小有收获，参与了Ray/vLLM社区，提了一些issue和pr😊，这个领域很合我胃口，后续可能也会focus这边继续深耕。

背景：公司内部用的ray是魔改的ray2.35.0，而业务部门有需求升级到最新版本的vllm，官方的setup已经要求依赖的ray的版本大于等于2.46，因此有必要测试一下这之间有多少Gap。通过这篇文章（更像是实验记录），你可以快速了解到Ray与vLLM的耦合程度，文中也给出了很多代码或链接提供快速验证。

声明：本文涉及的内容已经过筛选，确保全部来源于开源社区。另外由于是直接搬运个人给公司内部写的report，因此语言是英文，见谅。

Main source codes

vLLM set Ray as default multi-node executor backend, the main logic are as follows:

https://github.com/vllm-project/vllm/blob/main/vllm/executor/ray_distributed_executor.py

https://github.com/vllm-project/vllm/blob/main/vllm/executor/ray_utils.py

Below are the core features of Ray has been used by vLLM:

- Ray Actors / Workers
- Ray DAG (Compiled Graphs)
- Ray Cluster (Placement Groups, Resources...)

Use Case Limitations

1. Ray Data LLM API

<https://docs.ray.io/en/latest/data/working-with-llms.html#vllm-llm>

https://docs.vllm.ai/en/latest/serving/offline_inference.html?h=ray#ray-data-llm-api

```

import ray # Requires ray>=2.44.1
from ray.data.llm import vLLMEngineProcessorConfig, build_llm_processor

config = vLLMEngineProcessorConfig(model_source="unsloth/Llama-3.2-1B-Instruct")
processor = build_llm_processor(
    config,
    preprocess=lambda row: {
        "messages": [
            {"role": "system", "content": "You are a bot that completes unfinished haikus."},
            {"role": "user", "content": row["item"]},
        ],
        "sampling_params": {"temperature": 0.3, "max_tokens": 250},
    },
    postprocess=lambda row: {"answer": row["generated_text"]},
)

ds = ray.data.from_items(["An old silent pond..."])
ds = processor(ds)
ds.write_parquet("local:///tmp/data/")

```

2. Batch LLM Inference (also due to lack of ray data LLM api)

https://docs.vllm.ai/en/latest/examples/offline_inference/batch_llm_inference.html?h=ray

3. Ray Serve LLM API

https://docs.ray.io/en/latest/ray-overview/examples/e2e-rag/notebooks/03_Deploy_LLM_with_Ray_Serve.html#deploy-llm-with-ray-serve-llm

```

from ray import serve
from ray.serve.llm import LLMConfig

llm_config = LLMConfig(
    model_loading_config={
        'model_id': 'Qwen/Qwen2.5-32B-Instruct'
    },
    engine_kwargs={
        'max_num_batched_tokens': 8192,
        'max_model_len': 8192,
        'max_num_seqs': 64,
        'tensor_parallel_size': 4,
        'trust_remote_code': True,
    },
    accelerator_type='L4',
    deployment_config={
        'autoscaling_config': {
            'target_ongoing_requests': 32,
            'target_num_ongoing_requests_per_replica': 32,
        },
        'max_ongoing_requests': 64,
    },
)

```

4. Ray Compile graph

Ray version required 2.43.0

https://github.com/vllm-project/vllm/blob/main/vllm/executor/ray_distributed_executor.py#L517

```
required_version = version.parse("2.43.0")
current_version = version.parse(importlib.metadata.version("ray"))
if current_version < required_version:
    raise ValueError(f"Ray version {required_version} is "
                    f"required, but found {current_version}")

import importlib.util
cgraph_spec = importlib.util.find_spec(
    "ray.experimental.compiled_dag_ref")
```

which errors will occur:

```
ERROR 07-31 04:43:09 [engine.py:165] TypeError: DAGNode.experimental_compile() got an unexpected
keyword argument '_overlap_gpu_communication'
```

Related vllm PR: <https://github.com/vllm-project/vllm/pull/13994>

Related ray feature: <https://docs.ray.io/en/latest/ray-core/compiled-graph/ray-compiled-graph.html> <https://docs.ray.io/en/latest/ray-core/compiled-graph/overlap.html#experimental-overlapping-communication-and-computation>

It is a gap yet, we could set `VLLM_USE_RAY_COMPILED_DAG=0` to avoid use this feature.

Tested Cases (Passed)

Basic use with ray job (tensor_parallel_size > 1)

```
from vllm import LLM
llm = LLM("facebook/opt-125m", tensor_parallel_size=2)
output = llm.generate("San Francisco is a")
```

Vllm serve using ray

```
python3 -m vllm.entrypoints.openai.api_server --port 8080 --model Qwen/Qwen2.5-1.5B-Instruct --
tensor-parallel-size 2 --distributed-executor-backend=ray
```

Quick start: https://docs.vllm.ai/en/latest/getting_started/quickstart.html

Multi-Node Serving: https://docs.vllm.ai/en/latest/examples/online_serving/multi-node-serving.html

RLHF using ray: https://docs.vllm.ai/en/latest/examples/offline_inference/rlhf.html?h=rlhf

Related Discussion

- <https://github.com/vllm-project/vllm/issues/20860>
- <https://github.com/vllm-project/vllm/issues/20476>